Candidate Chemical-Formula Generation for Three Material Systems

Methodology, reference datasets, novelty filtering and diversity selection

CrystalFNO discovery pipeline
`crystalfno-discovery` (`Chemical-Formula-Parser`)

21 September 2026

Abstract

We describe the pipeline used to generate 1000 candidate chemical formulas for each of three composition spaces — oxychalcogenides, phosphorus chalcogenides and double perovskites. Candidates are produced by an exact, deterministic enumeration of charge-balanced integer stoichiometries over a `config.yaml`-declared element universe, screened for chemical plausibility with SMACT 4.0.0 (charge neutrality under the ICSD-2024 oxidation-state set plus the Pauling electronegativity test), filtered against the union of the Crystallography Open Database, the Materials Project and the Open Quantum Materials Database (3,597,064 unique known reduced formulas), and reduced to a maximally spread-out subset by greedy farthest-point (max–min dispersion) sampling over a combined stoichiometric and element-identity distance. All reported numbers are measured from the delivered runs. The three novelty-filtered outputs contain 1000 formulas each, with zero duplicates, zero parenthesised groups, zero SMACT-invalid compositions and zero overlap with the reference databases.

Contents

1 Summary **2**

2 The three composition spaces **2**

3 Candidate generation algorithm **3**

4 Chemical-plausibility screening with SMACT **3**

5 Reference datasets used for novelty filtering **4**

6 How novelty filtering is done **5**

7 How diversity is achieved **6**

7.1 Stratified enumeration (pool construction) 6

7.2 Distance metric 6

7.3 Selection algorithm: greedy farthest-point sampling 6

7.4 Measured effect 7

8 Output notation **7**

9 Reproducibility **7**

10 Limitations and scientific safeguards **8**

1 Summary

For each composition space the pipeline has four stages:

(1) **Enumerate** charge-balanced compositions from the config-declared element universe (cation set × anion set × integer stoichiometry), stratified so that no single element combination dominates the pool.
(2) **Screen** every candidate with SMACT [5, 6] (`smact.screening.smact_validity`): charge neutrality under an oxidation-state set *and* the Pauling electronegativity rule [7].
(3) **Filter** out any formula whose canonical reduced form already occurs in COD [1], MP [2, 3] or OQMD [4].
(4) **Select** a maximally spread-out subset of the survivors with greedy farthest-point sampling [8] over a combined L2 / Jaccard [9] distance.

Output is written one formula per line in flat, parenthesis-free notation (`La4S3O3`, not `La4(SO)3`).

System	Pool	SMACT rej.	Known in DBs	Novel	Elem. sets	Min d
Oxychalcogenides	14,975	0 (0.0 %)	830 (5.5 %)	14,145	1,000	0.5354
Phosphorus chalcogenides	6,768	36 (0.5 %)	81 (1.2 %)	6,651	898	0.2720
Double perovskites	11,019	0 (0.0 %)	2,088 (18.9 %)	8,931	1,000	0.4905

Table 1: Statistics of the three delivered runs ($n = 1000$, novelty filter on, diversity selection on). "Min d" is the minimum pairwise distance of the delivered selection.

Verification performed on the six output files: 1000 lines each; 0 parenthesised groups; 0 duplicate formulas; 0 formulas failing the SMACT validity test; and, for the three `novel_1000.txt` files, 0 canonical forms present in the 3,597,064 known formulas. The three `candidates_1000.txt` comparison sets are diversity-selected but deliberately *not* novelty-filtered, so they do contain known formulas (70, 16 and 115 respectively).

2 The three composition spaces

The chemistry of each system is declared entirely in `config.yaml` under `composition_spaces`; no element list is hard-coded in Python. Every space specifies a cation pool, an anion pool, per-element oxidation states used for charge balancing, and enumeration constraints (Table 2).

Chemical rationale for the caps: rare-earth/transition-metal oxychalcogenides (photocatalysis, thermoelectrics, ion storage) are mostly reported with at most three distinct cation species; layered transition-metal phosphorus chalcogenides (e.g. `TiPS3`, `FeCoPS`) are one-to-four cation systems; and the $A_2BB'O_6$ double-perovskite motif contains at most four distinct cation species and at most ten atoms per formula unit.

`max_cation_species` matters for tractability as well as chemistry: without it the double-perovskite space walks all 2^{19} cation subsets and one run took 272 s; capped at four the same run takes 5 s.

Space	Cations	Anions	Constraints
`oxychalcogenides`	La, Ce, Pr, Nd, Sm, Eu, Gd, Ti, V, Cr, Mn, Fe, Co, Ni (14)	O, S, Se, Te; O required	max_stoich 8, max_total_atoms 20, max_cation_species 3, per_combo_limit 4
`phosphorus_chalcogenides`	Ti, V, Cr, Mn, Fe, Co, Ni, Cu (8)	P, S, Se, Te; P required	max_stoich 8, max_total_atoms 20, max_cation_species 4, per_combo_limit 6
`double_perovskites`	La, Nd, Sm, Eu, Gd, Ca, Sr, Ba, Ti, V, Cr, Mn, Fe, Co, Ni, Cu, Nb, Mo, W (19)	O	max_stoich 6, max_total_atoms 10, max_cation_species 4, per_combo_limit 6

Table 2: Composition spaces as declared in `config.yaml`.

3 Candidate generation algorithm

Implementation: `src/composition.py` (`CompositionSpace`, `generate_charge_balanced_stoichiometries`, `check_charge_balance`).

For each pair (cation subset C, anion subset A) satisfying

$$\begin{aligned} &\text{min_cations} \leq |C| \leq \text{max_cation_species}, \\ &\text{min_anions} \leq |A| \leq \text{max_anion_species}, \\ &\text{required_anions} \subseteq A, \end{aligned}$$

the pipeline performs the following steps:

1. Enumerate integer stoichiometry vectors over $C \cup A$ with each coefficient between 1 and `max_stoich` and total atom count at most `max_total_atoms`. A work budget (`max_scan`, default 20,000 vectors per combination) bounds the cost of large combinations.
2. For each vector, test whether an assignment of oxidation states — taken from the per-element lists in `config.yaml` — makes the composition exactly charge neutral. Only vectors admitting such an assignment survive.
3. Canonicalise the surviving composition with `formula_utils.normalize_formula` (pymatgen [10] `Composition.get_reduced_formula` semantics) and deduplicate, so that `AlCrNb2` and `Nb2CrAl` collapse to one entry.
4. Keep at most `per_combo_limit` formulas per (C, A) combination (stratification, Section 7.1) and stop at `max_results` overall.

The enumeration is exact and deterministic: it neither samples randomly nor depends on dictionary ordering, so re-running a script reproduces byte-identical output.

4 Chemical-plausibility screening with SMACT

Library: SMACT (*Semiconducting Materials by Analogy and Chemical Theory*), version 4.0.0 [5, 6]. Wrapper: `src/smactlib/filters.py`; CLI wiring: `src/candidate_cli.py` (`_smact_validator`, `_screen`).

Every enumerated candidate is passed through

```
smact.screening.smact_validity(formula,
                               use_pauling_test=True,
                               include_alloys=True,
```

`oxidation_states_set="icsd24")`

so a candidate is kept only if SMACT can find an oxidation-state assignment, drawn from the ICSD-derived 2024 oxidation-state table shipped with SMACT, that

(a) sums to zero charge for the given stoichiometry, and
(b) obeys Pauling's electronegativity rule [7]: negative oxidation states are assigned only to the more electronegative elements of the composition.

All-metallic compositions are auto-accepted as alloys (`include_alloys=True`), which is SMACT's convention for intermetallics where integer charge balancing is not meaningful.

Because the `config.yaml` enumeration already enforces charge neutrality with its own oxidation-state lists, SMACT acts here mainly as an independent cross-check that additionally imposes the electronegativity constraint. The measured rejection rates confirm that the two are largely consistent:

$$\text{oxychalcogenides } \frac{0}{14{,}975}, \qquad \text{P-chalcogenides } \frac{36}{6{,}768}\ (0.5\%), \qquad \text{double perovskites } \frac{0}{11{,}019}.$$

The 36 rejected phosphorus-chalcogenide candidates are Te-rich compositions such as `TiTe4P`, `VTe5P`, `CrTe7P2` and `Cr(Te4P)2`. Pauling electronegativities put P (2.19) marginally above Te (2.10), so the only charge-balancing assignment for these formulas requires Te to carry the negative state and P the positive one, which the Pauling test forbids. Screening costs 0.8 ms per formula and is applied to the whole pool, i.e. a few seconds per run. It is on by default and can be retuned from the command line: `-no-smact`, `-smact-ox-set {icsd24,icsd16,smact14,pymatgen_sp,wiki}`, `-smact-no-en`.

SMACT as an independent generator (complementary path)

`src/smactlib/generator.py` also implements full SMACT-driven candidate generation: for every element combination of the requested size it calls `smact.neutral_ratios` over the SMACT oxidation-state sets, optionally applying `smact.screening.pauling_test`, and enumerates integer stoichiometry vectors for all-metal (alloy) combinations. That path is driven by `scripts/generate_formulas.py` (which writes `data/formulas/candidates.csv` with per-candidate SMACT flags) and screened for novelty by `scripts/find_novel_formulas.py`. It searches an arbitrary element universe rather than a cation/anion-structured space, and is therefore complementary to the three system-specific pipelines. The 3,000 delivered formulas were produced by the config-driven enumerator of Section 3 and validated by the SMACT screen described above.

5 Reference datasets used for novelty filtering

Three databases were used, each mirrored locally under `~/Git/<name>-Dataset` and reduced to a list of unique canonical formulas in `datasets/` by `scripts/extract_database_formulas.py`. That extraction is run once; the CSVs are the reusable artefact, and `datasets/` is intentionally `.gitignore`d because of its size.

Cross-database overlap within the union: COD ∩ MP = 25,971; COD ∩ OQMD = 47,141; MP ∩ OQMD = 75,399; present in all three = 25,556. Hence 122,955 formulas are shared by at least two databases and the union is 3.3 % smaller than the sum.

Composition of the union by number of distinct elements: 1 element, 104; 2 elements, 185,555; 3 elements, 842,541; 4 elements, 1,973,133; 5 elements, 395,768; 6 elements, 128,402; 7 or more, ≈71,000.

Database	Source snapshot	Unique formulas	Size
COD [1]	rsync mirror of `crystallography.net/cif` (experimental crystal structures)	399,556	6.4 MB
MP [2, 3]	`data/mp_summary.csv` via the Materials Project API (DFT-computed entries)	104,734	1.1 MB
OQMD [4]	SQL dump `qmdb_v1_8`, release 2026-01-29, `compositions` table (DFT-computed entries)	3,215,729	35.4 MB
	Sum	3,720,019	
	Union (deduplicated)	3,597,064	

Table 3: Reference databases and the extracted known-formula sets.

Extraction details worth recording:

- Every formula is re-canonicalised with `normalize_formula` before being written, so the three CSVs share one notation and plain string comparison is a valid novelty test. Canonicalisation was verified to be notation- and element-order-independent (200 random element-order permutations per sample formula, 0 inconsistencies).
- COD formulas come from parsing the CIF mirror, MP from the summary CSV, and OQMD from the SQL dump. The OQMD dump packs many rows into a single `INSERT` statement, so extraction matches every `('formula',` tuple on `compositions` lines rather than only the first one per line; an early version that matched only the first recovered 181 formulas instead of 3.2 M.
- A small number of OQMD rows use placeholder site labels instead of element symbols. These fail element parsing and are dropped. In the unlikely case that a placeholder-only row used real element symbols it would survive as a spurious "known" entry; that can only cause false negatives (a genuinely novel formula reported as known), never false positives, and no such case was observed for the three heavy-cation systems studied here.

Regenerate the CSVs with:

```
conda run -n chem python scripts/extract_database_formulas.py
conda run -n chem python scripts/extract_database_formulas.py --db cod mp
```

6 How novelty filtering is done

Implementation: `src/novelty_filter.py` (`load_known_formulas_set`, `generate_novel_candidates`, `NovelGenerationResult`).

1. Load the union of the selected CSVs into a Python `set` of canonical strings (`-db cod mp oqmd` selects a subset; the default is all three). Loading 3,597,064 formulas takes a few seconds and dominates runtime.
2. Canonicalise each generated candidate with `normalize_formula` and test set membership. A candidate is *novel* if and only if its exact reduced formula is absent from every selected database.
3. In enumeration-order mode the generation limit is scaled adaptively (×4 per round, capped at 2,000,000) until the requested number of novel formulas is found, the composition space is exhausted, or the cap is hit. The result object reports `total_generated`, `rejected_by_validator`, `known_filtered`, `novel_found`, `space_exhausted` and `target_met`, so statistics are never inferred from a truncated list.

Novelty rates observed at pool level (fraction of charge-balanced, SMACT-valid candidates that no database contains) are given in Table 1: 94.5 % for oxychalcogenides, 98.8 % for phosphorus chalcogenides and 81.1 % for double perovskites. The lower double-perovskite rate is expected

— oxide perovskite compositions are the most heavily mined region of all three databases.

Scope. "Novel" here means *database-unseen at the level of the reduced chemical formula.* It is not a claim of thermodynamic stability, synthesizability, or that the material has never been made. The comparison is also composition-level only: two polymorphs of the same composition count as one known formula.

7 How diversity is achieved

Diversity is produced by two cooperating mechanisms: stratified enumeration, which makes the pool broad, and farthest-point selection, which makes the delivered subset spread out.

7.1 Stratified enumeration (pool construction)

Raw enumeration exhausts every stoichiometry of one element combination before moving to the next, and iterates cation subsets by increasing size, so the head of an unstratified list is dominated by a handful of species: in the first oxychalcogenide runs the first 1000 formulas spanned only 15 distinct element sets and were nearly all single-cation La systems. `per_combo_limit` caps how many formulas any single (cation-set, anion-set) combination contributes (4 for oxychalcogenides, 6 for the other two systems), spreading the pool across the whole combination space.

7.2 Distance metric

Implementation: `src/diversity.py`. Each formula is mapped to two 118-dimensional vectors over the periodic table: $\mathbf{f}$, the elemental mole fractions (summing to 1), and $\mathbf{p}$, the binary element-presence indicator. The distance between two formulas is

$$d(i,j) \;=\; w_{\text{comp}}\,\|\mathbf{f}_i - \mathbf{f}_j\|_2 \;+\; w_{\text{set}}\,\Big(1 - J(S_i,S_j)\Big), \qquad J(S_i,S_j) = \frac{|S_i \cap S_j|}{|S_i \cup S_j|}, \tag{1}$$

with J the Jaccard index [9] on element sets. The first term separates stoichiometries; the second separates chemical identity, so two formulas sharing no elements are pushed apart even when their fraction vectors look similar. Both weights default to 1.0 and are exposed as `-w-composition` and `-w-element-set` (e.g. `-w-element-set 3` emphasises distinct species over distinct stoichiometry).

7.3 Selection algorithm: greedy farthest-point sampling

`select_diverse(pool, n_select)` implements max–min dispersion [8]:

- deduplicate the pool and seed with the candidate farthest from the pool centroid (a deterministic, representative starting point);
- maintain $m_i = \min_{s \in \text{selected}} d(i,s)$ for every pool member i;
- repeatedly select $\arg\max_i m_i$ and update $\mathbf{m} \leftarrow \min(\mathbf{m},\, d(\cdot, i^*))$, marking selected entries as $-\infty$.

This is Gonzalez's 2-approximation for the max–min dispersion (p-center) objective [8]: the smallest pairwise distance in the returned set is at least half of the best achievable for that set size. Vectorised with NumPy it costs $O(|\text{pool}| \times n_{\text{select}})$ distance evaluations — reducing a 15,000-member pool to 1000 formulas takes a few seconds. The result is deterministic and reproducible.

7.4 Measured effect

	Baseline A no stratification, enumeration order	Baseline B stratified, enumeration order	Delivered stratified + FPS + novelty filter
Oxychalcogenides: distinct sets	15	258	1,000
min pairwise distance	0.0157	0.0288	0.5354
mean pairwise distance	—	1.2574	1.2975
P-chalcogenides: distinct sets	7	167	898
min pairwise distance	0.0157	0.0315	0.2720
mean pairwise distance	—	1.1824	1.1616
Double perovskites: distinct sets	155	228	1,000
min pairwise distance	0.0354	0.0354	0.4905
mean pairwise distance	—	1.1544	1.0689

Table 4: Diversity of 1000-formula outputs. Baseline A was measured on the pre-stratification pipeline; Baseline B is reproducible today with `-no-diverse`; "Delivered" is the content of the `novel_1000.txt` files.

Element coverage is complete in all delivered sets (18, 12 and 20 elements respectively — every element of the space appears), and the largest group of formulas sharing one element set drops from 4–6 to 1–3. Phosphorus chalcogenides reach 898 rather than 1000 distinct element sets because the space contains only 12 elements with P mandatory, so fewer than 1000 distinct sets are reachable at this size.

Honest caveat. Max–min dispersion optimises the *minimum* pairwise distance, not the mean. The mean can therefore fall slightly when the selection is forced to fill in isolated regions: for double perovskites it went from 1.1544 (Baseline B) to 1.0689 (delivered) while the minimum rose from 0.0354 to 0.4905 and distinct element sets rose from 228 to 1000. If average separation is the target instead, `-no-diverse` with a larger pool, or a max–sum objective, would be the right tool.

8 Output notation

`formula_utils.flatten_formula` expands parenthesised groups and re-emits the reduced composition with integer counts, keeping pymatgen's [10] conventional electronegativity-based element order (cation before anion): `La4(SO)3` becomes `La4S3O3`, `Ti(PS3)2` becomes `TiP2S6`. Unit counts are omitted (`LaTiSO2`, not `La1Ti1S1O2`). Flattening is notation-independent, idempotent and round-trips to the same canonical form as `normalize_formula` (covered by tests in `tests/test_formula_utils.py`), so flat output can still be matched against the parenthesised database CSVs by canonicalising both sides — which is exactly how the zero-overlap verification in Section 1 was performed.

9 Reproducibility

Each system has its own script; all share `src/candidate_cli.py`.

```
conda run -n chem python examples/oxychalcogenides/generate_candidates.py \
    -n 1000 --filter-known -o examples/oxychalcogenides/novel_1000.txt

conda run -n chem python examples/phosphorus_chalcogenides/generate_candidates.py \
```

```
    -n 1000 --filter-known -o examples/phosphorus_chalcogenides/novel_1000.txt

conda run -n chem python examples/double_perovskites/generate_candidates.py \
    -n 1000 --filter-known -o examples/double_perovskites/novel_1000.txt
```

The comparison sets drop `-filter-known` and write `candidates_1000.txt`. `examples/generate_candidates.py` runs all three spaces in one go, inserting the space name into the output filename.

Flags. `-n/-num`, `-o/-output`, `-filter-known`, `-db cod mp oqmd`, `-no-diverse`, `-pool`, `-per-combo`, `-max-cations`, `-max-anions`, `-w-composition`, `-w-element-set`, `-no-smact`, `-smact-ox-set`, `-smact-no-en`.

Cost. A full 1000-formula run (pool construction + SMACT screen + loading the 3.6 M-formula known index + FPS selection) takes 17 s wall clock; without `-filter-known` it is a few seconds. Output is deterministic.

Tests. `conda run -n chem python -m pytest -q` collects 100 tests, all passing, covering formula canonicalisation and flattening, charge-balance enumeration, the novelty and validator hooks, SMACT filtering and the diversity selector.

10 Limitations and scientific safeguards

- Charge neutrality plus the Pauling electronegativity rule are *necessary*, not sufficient, conditions for a real compound. They screen out impossible stoichiometries; they say nothing about thermodynamic stability, synthesis conditions or whether a phase can be isolated. No DFT or machine-learning stability prediction (e.g. formation energy above the convex hull) is applied at this stage.
- "Novel" means unseen in COD, MP and OQMD at the reduced-formula level. Absence from these databases is evidence of interest, not evidence of non-existence: the databases are incomplete, and unpublished or very recent compounds will be counted as novel.
- Oxidation states are taken from `config.yaml` for enumeration and from SMACT's `icsd24` set for screening. Systems with unusual or mixed-valence chemistry may need a different set (`-smact-ox-set`) or a config edit.
- Diversity is measured in composition space only. Structurally distinct polymorphs of one composition are a single point; conversely, two compositions far apart in metric (1) may still be structurally related.
- Per-combination work budgets (`max_scan`) and species caps (`max_cation_species`) mean the pool is a large, stratified sample of the space rather than a proof of exhaustive coverage.

Software availability

Pipeline code: `crystalfno-discovery` at `/home/huangyang/Git/Chemical-Formula-Parser` (modules `composition`, `diversity`, `novelty_filter`, `formula_utils`, `candidate_cli`, `smactlib`). Environment: conda env `chem` with SMACT 4.0.0, pymatgen, NumPy, SciPy, pandas, PyYAML. Reference-database extracts: `datasets/{cod,mp,oqmd}_formulas.csv`, regenerated by `scripts/extract_database_formulas.py`.

References

[1] S. Gražulis, D. Chateigner, R. T. Downs, A. F. T. Yokochi, M. Quirós, L. Lutterotti, E. Manakova, J. Butkus, P. Moeck, and A. Le Bail. Crystallography Open Database — an open-access collection of crystal structures. *Journal of Applied Crystallography*, 42(4):726–729, 2009. doi: 10.1107/S0021889809016690

[2] A. Jain, S. P. Ong, G. Hautier, W. Chen, W. D. Richards, S. Dacek, S. Cholia, D. Gunter, D. Skinner, G. Ceder, and K. A. Persson. Commentary: The Materials Project: A materials genome approach to accelerating materials innovation. *APL Materials*, 1(1):011002, 2013. doi: 10.1063/1.4812323

[3] Materials Project. Data accessed via the MP API; local snapshot `mp_summary.csv` (`~/Git/MP-Dataset`).

[4] S. Kirklin, J. E. Saal, B. Meredig, A. Thompson, J. W. Doak, M. Aykol, S. Rühl, and C. Wolverton. The Open Quantum Materials Database (OQMD): assessing the accuracy of DFT formation energies. *npj Computational Materials*, 1:15010, 2015. doi: 10.1038/npjcompumats.2015.10 Snapshot used: QMDB v1.8 SQL dump, release 2026-01-29.

[5] D. W. Davies, K. T. Butler, A. J. Jackson, J. M. Skelton, K. Morita, and A. Walsh. SMACT: Semiconducting Materials by Analogy and Chemical Theory. *Journal of Open Source Software*, 4(38):1361, 2019. doi: 10.21105/joss.01361 Version used: 4.0.0.

[6] D. W. Davies, K. T. Butler, A. J. Jackson, A. Morris, J. M. Frost, J. M. Skelton, and A. Walsh. Computational Screening of All Stoichiometric Inorganic Materials. *Chem*, 1(4):617–627, 2016. doi: 10.1016/j.chempr.2016.09.010

[7] L. Pauling. *The Nature of the Chemical Bond*. Cornell University Press, Ithaca, NY, 3rd edition, 1960. Electronegativity criterion implemented by `smact.screening.pauling_test`.

[8] T. F. Gonzalez. Clustering to minimize the maximum intercluster distance. *Theoretical Computer Science*, 38:293–306, 1985. doi: 10.1016/0304-3975(85)90224-5 Greedy farthest-point sampling; 2-approximation for max–min dispersion.

[9] P. Jaccard. The distribution of the flora in the alpine zone. *New Phytologist*, 11(2):37–50, 1912. doi: 10.1111/j.1469-8137.1912.tb05611.x

[10] S. P. Ong, W. D. Richards, A. Jain, G. Hautier, M. Kocher, S. Cholia, D. Gunter, V. L. Chevrier, K. A. Persson, and G. Ceder. Python Materials Genomics (pymatgen): A robust, open-source python library for materials analysis. *Computational Materials Science*, 68:314–319, 2013. doi: 10.1016/j.commatsci.2012.10.028 Used for formula parsing, reduction and conventional element ordering.